`random()` function will begin with the very same numbers every time. That can be a little predictable. To take this predictability out we need to use the `randomSeed()` function with a syntax as follows:

```
randomSeed(value)
```

By feeding the `randomSeed()` function a seed value, we can kick off the random generation at a more unexpected point somewhere in the depths of the random sequence. This seed value can be either an integer or long data type while the initial call to the `randomSeed()` function often happens in the `setup()` function although this is not entirely necessary. So, consider the following single line statement:

```
randomSeed(42);
```

This line is a reasonable example of using `randomSeed()` to start the sequence off in a little different direction than it would without it. However, with a fixed seed value, in this case the integer 42, the sequence will still repeat exactly the same each time the sketch is ran. This can be occasionally useful, but to make things even more random we can use a reading of a disconnected, or floating, analog pin as our seed value. An example of this would be the following line:

```
randomSeed(analogRead(A0));
```

Remember earlier in this book when we discussed how a floating pin is a bad thing? In this case the odd gibberish that we get from reading an unconnected analog input pin is just the thing we need to increase the "randomness" of our random numbers. By placing the `analogRead()` function inside of the `randomSeed()` function, and pointing it to an unconnected analog in pin, A0 in this case, we will get a more arbitrary starting point for our random sequence. This also has the benefit that every time the sketch starts over by turning on or being reset, or simply the next time we call the function in the same manner, we will usually get completely different results. Remember, though, that in order for this to function properly, the pin must be disconnected from any circuitry.

■ **Note** As of the summer of 2011, the Arduino Uno interface board is available in an SMD edition that features a smaller surface mount version of the ATmega328 microcontroller chip. A little undocumented bonus of this version of the chip is that it has an additional two ADC pins, A6 and A7. While these pins are not accessible on the Arduino pin headers, we can still use them for seed values because they are not connected to anything. The next time you need to use this function, try randomSeed(analogRead(A6)); if you have this version of the board.

That wraps up a few additional functions that we have not previously been able to talk about in a sufficient depth. As you probably have noticed, we use delays of one form or another a lot in our example code and randomness can come in handy on occasion. Let's move on from using built-in Arduino functions to writing our own by looking at our next project, which is followed by a discussion of writing and using functions.

Project 6: Ambient Temps

This project will provide a visual indication of the ambient temperature in our immediate location by pairing a simple temperature sensor with an RGB LED. You might want to embed this project in a glass jar, a lamp, or other object. When it gets hot, our LED will fade to a deep red color; when it's cold, a deep